

Company Brain

Module 4 - Knowledge Retrieval Engine

Engineering Notes (Response 2)

Table of Contents

1. Knowledge Validator
 2. Knowledge Gap Detector
 3. Evidence Quality Engine
 4. Semantic Retrieval Cache
 5. KnowledgeContext
 6. Complete End-to-End Request Lifecycle
 7. Module 4 Folder Structure
 8. Design Principles
 9. Production Considerations
 10. Complete Module 4 Summary
-

12. Knowledge Validator

Purpose

The Knowledge Validator ensures that all synthesized knowledge is **correct, secure, explainable, and supported by evidence** before it becomes part of the final KnowledgeContext.

Its goal is to answer one question:

Can this knowledge be trusted?

Why is it Needed?

Enterprise AI systems cannot rely on assumptions or unsupported conclusions.

Every important fact should have:

- Supporting evidence
- Correct permissions
- Valid graph relationships

- Tenant isolation
- Confidence score

Without validation:

- Hallucinations increase
- Security risks appear
- Trust decreases

Responsibilities

Source Validation

Every derived fact must have evidence.

Example

Derived Fact

Vendor delayed shipment.

Supporting Evidence

Slack Message

Email

Vendor Contract

If no evidence exists:

Status

UNVERIFIED

Relationship Validation

Verify graph consistency.

Correct

Project

↓

Owned By



Engineering Team

Incorrect

Project



Owned By



PDF Document

The validator rejects invalid relationships.

Tenant Isolation

Every object must belong to the requesting tenant.

Example

Tenant

Microsoft

Retrieved document

Amazon



Rejected

Permission Validation

Even inside one tenant:

Employee



Cannot view



Executive Salary Documents

Validator removes unauthorized evidence.

Duplicate Validation

Detect semantic duplicates.

Example

Slack



Delay

Email



Same delay



Merged

Confidence Validation

Example

Confidence

0.95

Verified

0.75

Likely

0.55

Weak

0.20

Rejected

Output

Validated knowledge ready for packaging.

13. Knowledge Gap Detector

Purpose

Instead of inventing answers,

Company Brain identifies missing knowledge.

Question answered:

What information is incomplete or missing?

Types of Knowledge Gaps

Missing Entity

Question

Who owns Vendor Contract?

Graph

Vendor Contract

↓

Owner

???

↓

Knowledge Gap

Missing Relationship

Documents mention

Project Alpha depends on Project Beta

Graph contains no dependency.

↓

Gap detected.

Missing Timeline

Question

What happened in March?

Timeline

January

↓

February

↓

April

↓

Gap

Missing Decision

Meeting says

Deployment postponed

Nobody recorded

Who approved?

↓

Gap

Missing Evidence

Derived fact exists

↓

No citations

↓

Gap

Contradictions

Slack

↓

Owner = Alice

Email

↓

Owner = Bob

↓

Ownership Conflict

Why Gaps Matter

Instead of hallucinating,

Company Brain reports

Owner Unknown

Evidence Missing

This makes the system transparent.

Output

Example

```
Python
KnowledgeGap

id

type

severity

entity

description

recommended_action
```

Run

14. Evidence Quality Engine

Purpose

Not all evidence has equal value.

The Evidence Quality Engine measures

How trustworthy is this evidence?

Quality Metrics

Freshness

Recent information is usually more valuable.

Example

Yesterday



0.95

Two Years



0.20

Authority

Example

CEO Signed Document



1.00

Slack Message



0.65

Consistency

Three sources agree



High score

Sources disagree



Lower score

Completeness

Project record

Contains

- Owner
- Budget
- Timeline
- Status



High completeness

Missing fields



Low completeness

Source Reliability

Different systems have different reliability.

Example

SAP ERP

0.98

Salesforce

0.96

Jira

0.92

Slack

0.70

Final Trust Score

Example

Freshness

0.91

Authority

0.95

Consistency

0.94

Reliability

0.93

Completeness

0.90

↓

Trust Score

0.93

Output

Every evidence group receives:

- Trust Score
- Quality Metrics
- Confidence

15. Semantic Retrieval Cache

Purpose

Avoid repeating expensive retrieval.

Without cache

100 users ask

Why is Project Phoenix delayed?

↓

100 retrieval pipelines.

With cache

First request

↓

Full retrieval

Remaining requests

↓

Cache hit

Why Semantic Cache?

Traditional cache

Question A

Why is Phoenix delayed?

Question B

Why has Project Phoenix been delayed?

Different text.

Traditional cache misses.

Semantic cache compares embeddings.

Meaning is identical.

↓

Cache hit.

Cache Key

Should include

- Tenant
- User permissions
- Query embedding
- Knowledge version

Never use only question text.

Cache Expiration

Knowledge changes.

Use TTL.

Example

30 Minutes

Cache Invalidation

Project status changes

↓

Old cache removed

↓

New retrieval performed

Cache Warming

Nightly jobs generate

Frequently requested KnowledgeContexts.

Morning requests become instant.

16. KnowledgeContext

Purpose

KnowledgeContext is the final product of Module 4.

It is **not** an English answer.

It is a structured knowledge package.

Why?

Every downstream module uses the same object.

Instead of repeating retrieval,
they consume KnowledgeContext.

Structure

Python

Run

KnowledgeContext

query

query_analysis

retrieval_plan

evidence

entities

relationships

timeline

decisions

risks

derived_facts

citations

confidence

quality

knowledge_gaps

Metadata

Includes

- Request ID
- Tenant
- Processing Time
- Cache Hit
- Timestamp
- Version

Useful for debugging.

Consumers

KnowledgeContext can be used by

Chat



Natural language answer

AI Agent



Planning

Dashboard



Charts

Workflow Engine



Automation

Knowledge APIs



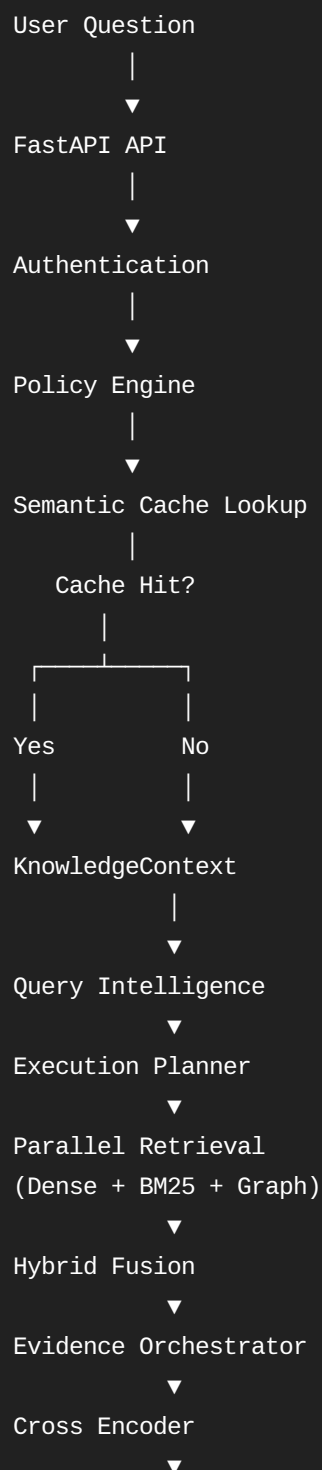
REST

GraphQL

MCP

17. Complete End-to-End Lifecycle

Complete pipeline



```
Knowledge Synthesizer
  ▼
Knowledge Validator
  ▼
Knowledge Gap Detector
  ▼
Evidence Quality Engine
  ▼
KnowledgeContext Packaging
  ▼
Semantic Cache Store
  ▼
Return KnowledgeContext
```

18. Module 4 Folder Structure

```
module4/

├─ api/
│   └─ search.py
│
├─ policy/
│   ├── tenant_policy.py
│   ├── acl.py
│   └─ permission_service.py
│
├─ intelligence/
│   ├── intent_classifier.py
│   ├── entity_extractor.py
│   ├── relationship_extractor.py
│   └─ query_analyzer.py
│
├─ planner/
│   ├── retrieval_planner.py
│   └─ planner_models.py
│
├─ retrievers/
│   ├── dense.py
│   ├── bm25.py
│   ├── graph.py
│   └─ hybrid.py
│
├─ fusion/
│   ├── rrf.py
│   └─ graph_priority.py
│
├─ orchestrator/
│   └─ evidence_orchestrator.py
│
```

```
|— reranker/
|   └─ bge_reranker.py
|
|— synthesizer/
|   └─ synthesizer.py
|
|— validator/
|   └─ validator.py
|
|— gap_detector/
|   └─ detector.py
|
|— quality/
|   └─ quality_engine.py
|
|— cache/
|   └─ semantic_cache.py
|
|— models/
|   └─ query_analysis.py
|   └─ retrieval_plan.py
|   └─ evidence.py
|   └─ knowledge_context.py
|
|— services/
|   └─ retrieval_service.py
```

19. Design Principles

Module 4 follows these principles:

Separation of Concerns

Each component has one responsibility.

Stateless Processing

No request-specific state stored in services.

Explainability

Every fact links back to evidence.

Auditability

Every decision can be traced.

Tenant Isolation

No cross-company access.

Reusability

KnowledgeContext works for every downstream module.

Scalability

Parallel retrieval

Semantic cache

Horizontal scaling

Security

ACL

JWT

Row-Level Security

Policy Engine

20. Module 4 Summary

Purpose

Module 4 transforms a natural language question into a **trusted, structured, evidence-backed KnowledgeContext**.

Inputs

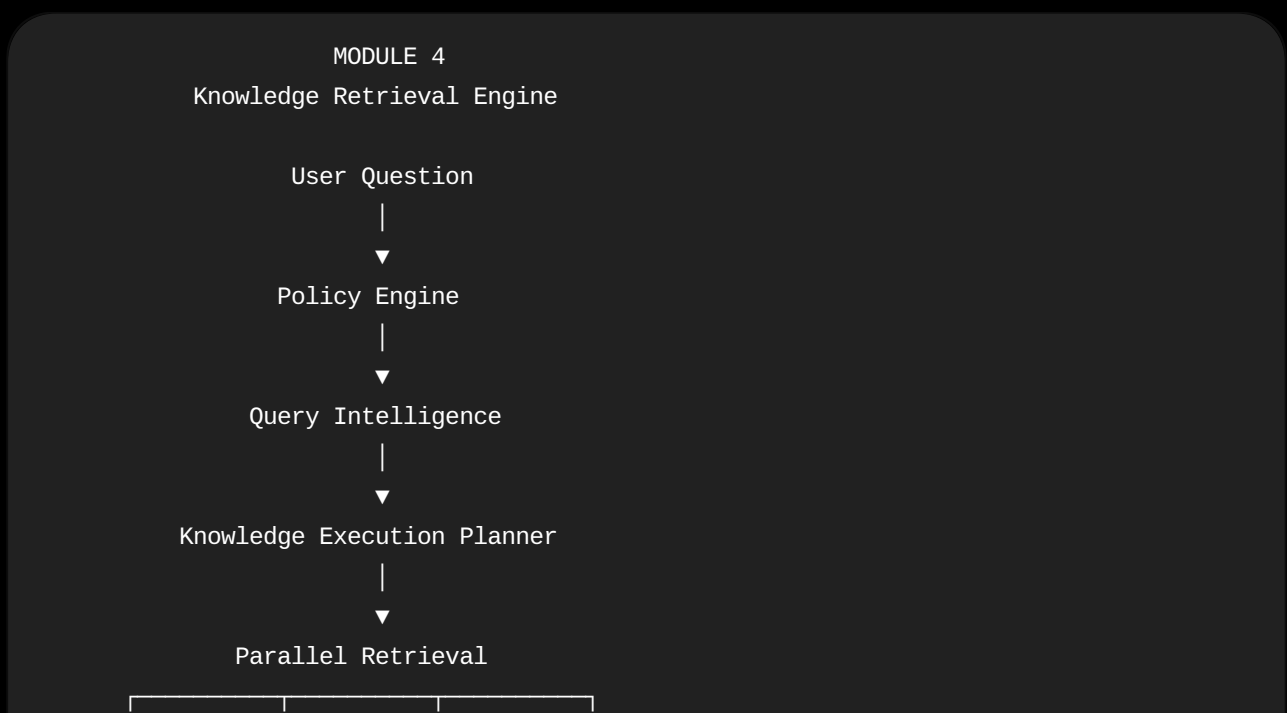
- User Question
 - User Identity
 - Tenant
 - Permissions
-

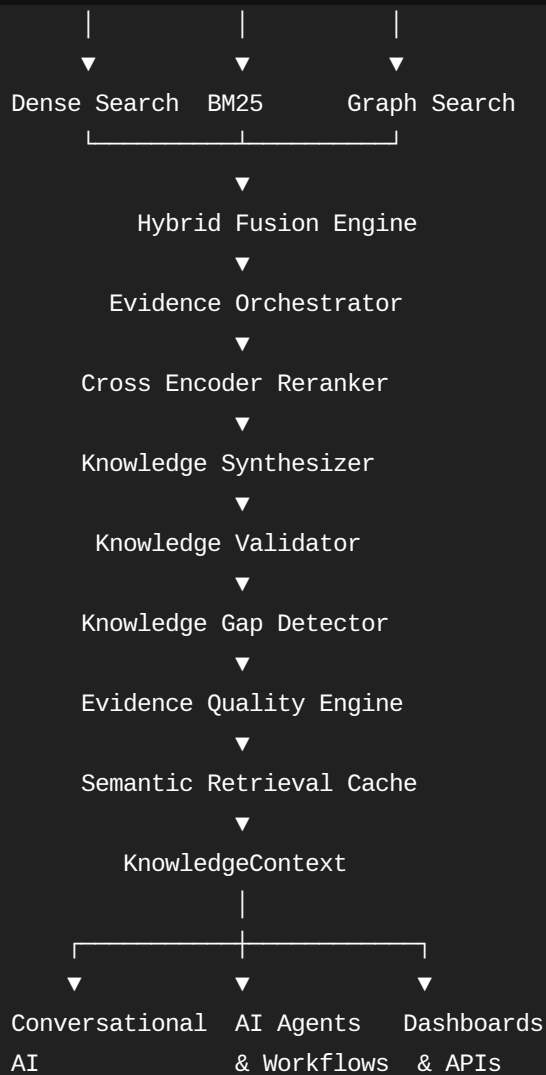
Outputs

KnowledgeContext containing:

- Query Analysis
 - Retrieval Plan
 - Evidence
 - Entities
 - Relationships
 - Timeline
 - Decisions
 - Risks
 - Derived Facts
 - Citations
 - Confidence
 - Trust Scores
 - Knowledge Gaps
 - Metadata
-

Complete Architecture





Module 4 in One Sentence

Module 4 is the intelligence layer of Company Brain that securely retrieves, ranks, validates, organizes, and packages enterprise knowledge into a reusable `KnowledgeContext`, enabling every downstream module to work from the same trusted foundation instead of performing its own retrieval and reasoning.